STRATHEX SWITCHYARD / SXSY-B122

SXSY-B122 Operator, Technical, and Troubleshooting Manual

SOFTWARE SCOPE	Hardware-independent source and local verification
ACCEPTANCE	All 27 gates are NOT ACCEPTED
LICENSING	Unresolved maintainer/legal decision
DOCUMENT	Version 1.1.0

A B122 chord expresses intent. It is not authority, authentication, confirmation, or permission to perform a side effect.

This manual includes the mechanically reconciled 22-control / 65-chord reference, first-install and lifecycle procedures, operator review semantics, fail-closed recovery, and troubleshooting. It contains no hardware or provider acceptance claim.

> *Canonical sources: accepted complete specification, command catalog, protocol manifest, adversarial review, and acceptance-gate register.*

Contents

1. Read This First . . . 4
1.1 What this manual covers . . . 4
1.2 Three meanings that must stay separate . . . 4
1.3 Never infer these claims . . . 4
1.4 Hardware-independent software closure evidence . . . 4
2. System at a Glance . . . 5
2.1 Control path . . . 5
2.2 Trust boundaries . . . 6
2.3 Session capabilities . . . 6
3. Installation and First Health Check . . . 6
3.1 Supported software-only boundary . . . 6
3.2 Build a release candidate . . . 7
3.3 Install from an independently reviewed bootstrap . . . 7
3.4 Start and health-check . . . 7
3.5 Developer source start . . . 7
3.6 Linux build, install, and health . . . 8
4. First VS Code Setup . . . 8
4.1 Verify the companion boundary . . . 8
4.2 Trust-changing configuration . . . 9
5. Daily Operator Workflow . . . 9
5.1 Before invoking a control . . . 9
5.2 Dispatch and review . . . 9
5.3 Authority Refresh and broadcast . . . 9
5.4 Durable operation status and cancellation . . . 10
5.5 Typed outcomes . . . 10
5.6 Automation review packets . . . 10
5.7 Alt-Run exact debug and managed-process stop . . . 11
6. Control Families and Consequence Boundaries . . . 11
6.1 Agent Control Plane . . . 11
6.2 Workspace Command Bar . . . 12
7. High-Consequence Controls . . . 13
8. Update, Rollback, Uninstall, and Logs . . . 13
8.1 Update . . . 13
8.2 Rollback . . . 13
8.3 Uninstall product bytes while preserving state . . . 14
8.4 Purge protected state . . . 14

8.5 What to preserve for support . . . 14
9. Firmware and Physical Device Boundary . . . 14
10. Recovery Rules . . . 15
10.1 Interrupted external work . . . 15
10.2 Re-anchor versus rollback . . . 15
10.3 Containment incident . . . 15
11. Troubleshooting . . . 15
11.1 Companion is unavailable . . . 15
11.2 VS Code shows no target or an ambiguous target . . . 16
11.3 A route is unavailable . . . 16
11.4 An operation will not cancel . . . 16
11.5 A target changed during confirmation . . . 16
11.6 A high-consequence action is blocked . . . 17
11.7 Tests must not touch production data . . . 17
12. Deterministic Safe Demo . . . 17
13. Evidence and Traceability . . . 18
14. Remaining External and Governance Work . . . 18
14.1 Physical or target-host prerequisites . . . 18
14.2 Licensing and release authority . . . 18
14.3 Historical records . . . 19
15. Acceptance Gate Register . . . 19
Appendix A. SXSY-B122 Printable 22-Control / 65-Chord Reference . . . 20
Legend sheet . . . 20
Exact chord reference . . . 21
Intentionally unassigned surface . . . 27
Appendix B. Source Index . . . 27

1. Read This First

1.1 What this manual covers

This is the first-install, daily-operation, recovery, and troubleshooting guide for the SXSY-B122 Windows and Linux companions, VS Code extension, provider-adapter host, signal-only firmware source, and the protected local control plane. It also contains the mechanically generated printable reference for all 22 controls and 65 settled chord assignments.

The accepted product behavior is controlled by `SXSY-B122-complete-key-specification-and-audit.md`. The current gate statuses are controlled by `acceptance-gates.md`. The historical interview notes are explicitly reconstructed and are not a verbatim transcript.

1.2 Three meanings that must stay separate

Term	Meaning
Implemented mechanism	Source-controlled software exists with a typed, fail-closed result.
Local verification	Isolated tests or package checks exercised the mechanism without production data.
Acceptance evidence	Current, authenticated, target-scoped observation from the real device, host, provider, or external service. No gate is accepted in this repository.

The software can be complete as source while a route is unavailable on a host. For example, a provider connector may be implemented but still require a separately authorized provider module, account, credential reference, exact network route, and live acceptance. Missing prerequisites must remain `unavailable`, `incomplete`, or `blocked`.

1.3 Never infer these claims

- Registered does not mean Observable, Controllable, or Mediated.
- A commit hash alone is not a complete dirty-worktree identity.
- A `.venv` is dependency isolation, not a security sandbox.
- Emergency Containment blocks only wired broker paths, not arbitrary shells or already-running external work.
- A green local test does not prove GitHub, CI, a provider, a scanner, Windows Sandbox, Linux Bubblewrap policy, or the physical keyboard.
- A successful package build is not a signed distribution or an accepted installation.
- A quick-reference row or hardware chord never grants permission.

1.4 Hardware-independent software closure evidence

The frozen source closes the nine previously recorded ordinary software gaps. This is a source-reachability and local-verification statement, not acceptance: it does not prove a physical signal, a live provider/account, target-host isolation, an external side effect, or any of the 27 gates.

Area	Frozen source and regression evidence
Automation	Proposal-derived inert manifests are executed only by the installed host-local broker and effect executor. Capture verifies exact bytes and the host-observed target transition; Refresh additionally uses the reviewed private-index local Git commit path. Provider automation effect and recovery routes are denied; only `automation.proposal.*` remains. See `automation-installed-runtime.ts`, `local-automation-broker.ts`, `automation-effect-executor.ts`, and `installed-automation-e2e.test.ts`.
Plan transition	Plan Approval binds the immutable expected TargetSnapshot transition, observes durable provider child effects, and preserves contradiction or ambiguous settlement for reconciliation. See `workflow-context-preparation.ts`, `effect-target-transition.ts`, and `workflow-artifacts.test.ts`.
Python environment transition	Atomic `.venv` creation/rebuild completes before the authenticated host action; the extension re-attests canonical path, SHA-256, filesystem identity, size, and modification time before configuration and again before terminal creation. See `atomic-python-venv.ts` and `controller.test.ts`.
Alt-Run	The split-binding review selects either one exact debug effect or one exact durable managed Run child. Only a completed, committed, identity-matching stop retires it; ambiguity remains reconciliation-visible. See `debug-stop.ts`, `repository-runtime-broker.ts`, and `controller.test.ts`.
Voice	All four Voice chords use one durable host-action claim, authorization, exact-target/session binding, and canonical settlement lifecycle; race, restart, stale-provider, cancellation, and cross-window cases fail closed. See `host-actions.ts` and `controller.test.ts`.
Target routes	Shift Terminal and blocked/finished queues preserve the exact repository; companion ownership is canonical; repeated Normal VS Code cycles only matching registered windows. See `controller.ts` and `controller.test.ts`.
Pipeline failure selection	Local and hosted observations retain honest unavailable/stale distinctions, and Normal/Alt share the protected selected failure index and series. See `pipeline-adapters.ts` and `controller.test.ts`.
Re-anchor policy	Generated policy includes exact provider-specific apply and observe routes; the provider host supports bounded stop/report/no-resume plus no-replay observation. See `New-B122NetworkPolicy.ps1` and `reanchor.test.ts`.
Refusal settlement	Physical second-phase Plan, automation, managed-stop, Authority Broadcast, and generic post-authorization failures preserve one canonical terminal hardware/effect disposition. See `controller.test.ts` and `hardware-signal-dispatcher.test.ts`.

Full `lfg` and Repair Mission are intentionally gate-disabled. Their typed plans, effect boundaries, and refusal behavior are implemented, but execution requires one current target/policy-bound activation, freshly re-read acceptance evidence, and every configured exact child artifact/backend. With all 27 gates still **NOT ACCEPTED**, the correct installed result is `blocked` or `unavailable`; that fail-closed result is not an ordinary software gap.

2. System at a Glance

2.1 Control path

```
Physical B122 or simulated signal
            |
            | versioned control + modifier only
            v
Windows or Linux companion and protected control plane
            |
            +-- authenticated named pipe (Windows) or Unix socket (Linux)
            +-- exact target/session/authority registry
            +-- durable operations, receipts, budgets, and containment
            +-- VS Code extension and operator review surfaces
            +-- installed typed brokers and provider adapters
```

Firmware is signal-only. It cannot inspect Git, choose a repository, infer an agent state, submit a prompt, approve a plan, or call GitHub. The companion owns policy and durable operation state. A typed adapter or broker owns the final effect, but only after exact target, session, authority, review, capability, containment, budget, and gate checks applicable to that action.

2.2 Trust boundaries

Boundary	Trusted for	Not trusted for
Firmware / Raw HID	One bounded control signal and transport identity.	Authority, target, confirmation, or gate evidence.
VS Code extension	Operator display, reviewed selection, local window/terminal identity.	Self-granting capabilities or bypassing the companion.
Companion	Protected state, routing, policy, durable operations, and exact reviews.	Claiming control over an unmediated process or external service.
Provider / repository / logs	Bounded data and typed observations after validation.	Instructions that alter authority, target, recipient, route, or gates.
Broker / installed adapter	One typed, permit-bound action with a final-current guard.	Free-form shell text, ambient credentials, force semantics, or fallback providers.

A provider module never receives an OS-opened credential, credential bearer, host endpoint, or companion broker. A separately configured, digest-pinned host effect driver is isolated from the provider module and receives only its fixed host identity plus the companion-issued bounded decision. Provider capability is capped below Mediated unless both that driver and a companion-authenticated, final-current callback are installed and exact-bound to the operation, action, provider session, authority, target, repository root, lease, deadline, and cancellation state.

2.3 Session capabilities

- **Registered:** the companion can identify or focus the surface.
- **Observable:** a structured adapter reports heartbeat and verified state.
- **Controllable:** the adapter can prove pause, context injection, or resume.
- **Mediated:** effects for the relevant operation pass through an enforceable broker or operating-system boundary.

Capabilities are independent facts. The extension initially registers a VS Code surface as Registered only. A fresh exact-target provider handshake is required before any stronger capability is recorded.

3. Installation and First Health Check

3.1 Supported software-only boundary

The verified release workflows are per-user on Windows and systemd-based x86-64 Linux. Windows product bytes live below:

```
%LOCALAPPDATA%\STRATHEX\switchyard\install\versions\<source-commit>
```

Protected state lives below:

```
%LOCALAPPDATA%\STRATHEX\switchyard
```

Linux versioned bundles and protected state live below:

```
$HOME/.local/share/strathex-switchyard/versions/<bundle-id>
${XDG_STATE_HOME:-$HOME/.local/state}/STRATHEX/switchyard
```

Repository files cannot redirect either location. A custom state root is for isolated testing only and cannot be combined with the installed VS Code extension.

3.2 Build a release candidate

Use the exact reviewed inputs required by `windows-release-and-installation.md`. The builder rejects a dirty worktree and requires a reviewed official Node Windows x64 archive plus digest. Native helper inputs are also digest-bound. First run `Provision-B122DotnetToolchain.ps1` on a clean checkout. It verifies the exact SDK archive and offline NuGet packages pinned by `dotnet-toolchain.json`; the release builder does not accept ambient .NET.

```
.\deployment\windows\Provision-B122DotnetToolchain.ps1

.\deployment\windows\Build-B122Release.ps1 `
  -NodeRuntimeZip C:\reviewed\node-v24-win-x64.zip `
  -NodeRuntimeSha256 <reviewed-sha256> `
  -NodeRuntimeSourceUri <reviewed-node-source-uri> `
  -SandboxHelperPath C:\reviewed\b122-sandbox-worker.exe `
  -SandboxHelperSha256 <reviewed-sha256> `
  -RawHidHelperPath C:\reviewed\strathex-b122-hid-helper.exe `
  -RawHidHelperSha256 <reviewed-sha256> `
  -ProviderDesktopHelperPath C:\reviewed\sxsy-b122-provider-desktop-helper.exe `
  -ProviderDesktopHelperSha256 <reviewed-sha256> `
  -VoiceHelperPath C:\reviewed\sxsy-b122-voice-helper.exe `
  -VoiceHelperSha256 <reviewed-sha256>
```

Do not use `-SkipVerification` for an installable candidate. It produces a development-only bundle that installation and startup correctly reject.

3.3 Install from an independently reviewed bootstrap

The bootstrap script and the expected trust digest/source commit are the initial trust anchor. Obtain them through the reviewed channel, then install from the candidate bundle:

```
.\Bootstrap-B122Install.ps1 `
  -BundleRoot C:\reviewed\SXSY-B122-release `
  -ExpectedTrustDigest <reviewed-64-character-sha256> `
  -ExpectedSourceCommit <reviewed-40-character-commit> `
  -OperationKindOverride install
```

Do not execute `Install-B122.ps1` directly for first install or external update. The bootstrap privately stages and verifies the trust-root documents before the first installation effect.

3.4 Start and health-check

The default install root contains the authenticated launcher. It repairs an interrupted install/update/rollback before starting the current version:

```
& "$env:LOCALAPPDATA\STRATHEX\switchyard\install\Invoke-B122Current.ps1" `
  -WaitForHealth
```

A healthy result requires the mutually authenticated local named-pipe probe and the exact selected payload identity. Process existence by itself is not health. The companion writes bounded stdout/stderr logs to:

```
%LOCALAPPDATA%\STRATHEX\switchyard\logs\companion.stdout.log
%LOCALAPPDATA%\STRATHEX\switchyard\logs\companion.stderr.log
```

If health fails, do not delete the state root or endpoint record. Follow the troubleshooting flow in section 11.

3.5 Developer source start

For source development only:

```
npm run build:companion
node .\apps\windows-companion\dist\apps\windows-companion\src\main.js
```

This is not a substitute for the verified Windows bundle. The npm package does not contain every native helper and is not release evidence.

3.6 Linux build, install, and health

Follow `linux-release-and-installation.md`. The Linux builder requires the reviewed official Node v24.14.0 x64 archive and its exact SHA-256, builds a real VSIX, writes a complete `MANIFEST.sha256`, and packages systemd user units. Installation re-verifies the copied payload, atomically changes the current version, and requires `sxsy-b122.service` to be active. Update and rollback preserve the previous immutable bundle; uninstall retains protected state and versioned payloads for recovery.

```
deployment/linux/build-b122-linux-release.sh \
  /reviewed/node-v24.14.0-linux-x64.tar.xz \
  41cd79bb7877c81605a9e68ec4c91547774f46a40c67a17e34d7179ef11729df \
  artifacts/sxsy-b122-linux-x64
artifacts/sxsy-b122-linux-x64/deployment/install-b122-linux.sh \
  artifacts/sxsy-b122-linux-x64
systemctl --user status sxsy-b122.service
```

Linux startup requires private current-user XDG state/runtime bases, `/usr/bin/secret-tool`, `/usr/bin/flock`, `/usr/bin/bwrap`, `/usr/bin/pw-record`, `/usr/bin/pw-play`, `/usr/bin/espeak-ng`, and a current-user Secret Service session. The companion uses a mode-`0600` authenticated Unix socket and stores only a Secret-Service-wrapped master key. It does not fall back to TCP, plaintext secrets, an ambient helper, or a copy-only sandbox. Voice capture begins each provider turn at frame sequence zero; only gateway-redacted response text reaches the trusted local `espeak-ng` to PipeWire synthesis path, and helper termination is deadline-bounded.

4. First VS Code Setup

4.1 Verify the companion boundary

The extension reads only the companion-created canonical endpoint record and opens its DPAPI CurrentUser-sealed token on Windows or Secret-Service-sealed token on Linux. It has no endpoint setting, TCP fallback, stdio fallback, or workspace-controlled token.

In the VS Code Command Palette:

1. Run **SXSY-B122: Register/Refresh This VS Code Window**.
2. Run **SXSY-B122: Show Managed Sessions**.
3. Confirm the new workspace/window record is shown as Registered unless a separately installed provider handshake proves more.
4. Run **SXSY-B122: Show 65-Chord Quick Reference** and confirm the exact target shown in each repository-bound row.
5. If several Git roots or sessions match, choose the exact item in the picker. Cancel when identity is uncertain.

4.2 Trust-changing configuration

Repository runtime manifests, Windows Sandbox catalogs, Linux Bubblewrap recipes, Raw-HID devices, credential references, exact HTTPS network policy, provider applications, provider handshakes, scanner fixtures, and acceptance evidence are installed through their named `SXSY-B122:` commands. Each installation requires reviewed source material and current-user presence (Windows Hello or Linux polkit). A workspace file cannot silently install or replace one of these records.

Never install a manifest merely to make an unavailable chord look green. Review the canonical path, digest, repository binding, route, purpose, expiry, and recovery behavior first.

5. Daily Operator Workflow

5.1 Before invoking a control

1. Identify the intended repository and VS Code window.
2. Check the selected managed session and its proved capabilities.
3. Check for unrelated dirty work and confirm the exact task-owned scope.
4. Check containment and any active durable operation.
5. Use the quick reference to review modifier, action, consequence, target, capability, confirmation, and required gates.
6. Cancel the picker or review if any binding is ambiguous.

5.2 Dispatch and review

The extension routes only one of the 65 canonical assignments. An unassigned modifier is rejected. For an eligible action, the companion prepares a bounded review. The review is tied to the exact action, target snapshot, provider session, scoped authority, manifest, and one-use confirmation; durable operation identity is allocated and bound at dispatch.

Review is not approval unless the exact review explicitly asks for and records a one-time confirmation. Repository text, terminal output, provider output, or a keyboard event cannot provide that confirmation.

5.3 Authority Refresh and broadcast

Normal Authority Refresh replays one existing scoped grant to the active exact session. It cannot create or broaden authority. If action, target, task, plan, session, or pending request differs, the request stops.

Shift Authority Refresh is a two-phase broadcast:

1. Select only exact verified-blocked Claude or Codex recipients on the same repository and TargetSnapshot with Registered, Observable, and Controllable capabilities.
2. Review/edit or cancel one individualized immutable packet per recipient, including its recipient, exact session, task, plan, and pending-request summary.
3. Review the exact prepared artifact ID, artifact digest, recipient set, and `reviewScopeDigest`.
4. Provide fresh reviewed current-user confirmation (Windows Hello or Linux polkit) bound to that exact artifact ID, artifact digest, and `reviewScopeDigest` through the host UI.

5. Send only if every recipient is still the selected exact session and all target/authority bindings remain current.

Cancelling either phase sends nothing, creates no broader permission, and preserves the prepared reconciliation evidence. A hardware-originated request uses the same immutable references and is settled exactly once.

5.4 Durable operation status and cancellation

Long-running work is identified by a durable operation ID. Use the extension's operation status surface, not Task Manager, a guessed PID, or a repeated chord. Select only the row whose action, repository root, session, authority, and target match the intended work. A repeated identical chord must focus the existing operation instead of starting a duplicate.

Cancellation is a recorded request, not an assertion that external work stopped. After selecting **Cancel** for the exact operation, expect `cancelling` until the installed adapter observes a terminal result. If observation is unavailable or ambiguous, the operation remains visible and non-retryable. Do not dispatch a replacement until reconciliation reports a terminal state.

5.5 Typed outcomes

Outcome	Operator meaning	Next action
unavailable	Required target, adapter, recipe, credential, route, provider, or host feature is absent.	Install/review the exact prerequisite or leave unavailable. Do not choose a weaker fallback.
incomplete	Snapshot or evidence could not be proved complete or changed during review.	Refresh from the current exact target and review again.
blocked	Policy, containment, authority, gates, or a safety invariant denied the request.	Inspect the named reason. Never edit a documentation status to bypass it.
awaiting_confirmation	The exact review is prepared and no effect has run.	Confirm only if every displayed binding is correct; otherwise cancel.
running	The durable operation is active or awaiting external observation.	Focus its status. Do not repeat the chord.
cancelling	Cancellation is recorded but terminal observation is pending.	Wait for reconciliation or investigate the exact adapter.
failed / timed_out	No successful durable conclusion was recorded.	Reconcile possible external effects before retry.
completed	A registered component returned its typed result and the receipt was recorded.	Review evidence; completion alone does not pass an acceptance gate.

5.6 Automation review packets

The installed Command Palette titles are **SXSY-B122: Compound Capture**, **Compound Refresh**, **Compound Full Loop**, and **Repair Mission**. Eligible automation uses this two-phase review; the key or provider cannot skip it:

1. Select one exact Mediated provider session.
2. The companion returns an inert immutable operation manifest. Nothing in the manifest is permission to execute.
3. The trusted VS Code review displays the complete target, session, authority, task, plan, provider, expiry, and every action-specific path, digest, check, network, and test restriction.
4. Approve only the exact displayed review-scope digest. Cancelling keeps the operation inert.
5. The companion reauthenticates the current user and issues one confirmation for that manifest and review scope.
6. Dispatch remains bound to the originally selected provider session. A provider proposal is never authority and cannot replace any manifest field.

Compound Full Loop (`lfg`) and Repair Mission remain gate-disabled even when a manifest can be prepared. Missing provider mediation returns unavailable; it does not fall back to a different provider or an unmanaged shell.

5.7 Alt-Run exact debug and managed-process stop

SXSY-B122: Stop Managed Run cannot stop an arbitrary debugger, terminal, or PID. Select one exact running B122-bound debug effect. Phase 1 sends only the provider origin as the authority session plus the debug managed-session ID to authenticated companion preparation. The review intentionally omits the protected effect-session ID and displays the exact provider source, authority, target, repository, origin operation, debug session, binding, configuration, and expiry.

Phase 2 echoes only the binding ID, binding digest, review scope, and provider source. Dispatch stays bound to that provider source; the companion recovers the effect-session ID from protected state, and the extension revalidates and stops only that exact current debug object. The keyboard or client never selects authority or an effect target.

No arbitrary or unmanaged debugger stop is supported. The same protected split-binding lifecycle can select the exact durable managed process tree previously launched by Run. A completed and committed response must repeat the managed child, effect session, and origin-operation identities before the extension retires that run. Missing or conflicting identity leaves the run visible for reconciliation and does not select another repository or PID.

6. Control Families and Consequence Boundaries

6.1 Agent Control Plane

- **Re-anchor** selects only an existing eligible authority grant for one exact managed session, shows a read-only exact-scope preview, and accepts only the modal **Confirm exact Re-anchor without resuming** choice. It then safely interrupts, rereads authority/context/docs, inspects work, and reports discrepancies. It cannot create or broaden authority and never silently resumes or rolls work back; cancellation sends no provider effect. Generated policy contains the exact provider-specific apply and no-replay observe routes; missing live provider configuration still returns unavailable.
- **Proof** evaluates evidence. Shift requests a separately selected cross-backend adversarial review. Neither form fixes, commits, or ships.
- **Option Provider** returns five choices plus one recommendation for one decision. Shift produces the pending-decision packet. Recommendations never choose or approve.
- **Plan Expander** returns fifteen ranked alternatives and does not replace the approved plan.
- **Plan Approval** starts only the exact approved revision in the already bound executor and requires exact authority/review. It cannot authorize unrelated actions. The immutable review includes the expected TargetSnapshot transition, and durable child observation preserves contradictory or ambiguous results for reconciliation.
- **Handoff** transfers an immutable context package. Queue, immediate focus, and routing-default changes do not execute or approve its contents.
- **Find Blocked/Finished** acts only on provider-verified states. Unknown, quiet, stale, or merely visible sessions are never relabelled. Shift queue pickers expose only exact-repository matches.
- **Authority Refresh** replays existing authority. Broadcast requires exact per-recipient artifact review. Cancellation, refusal, binding churn, and host failure settle once through the canonical hardware/effect receipt

path.

- **Council** Normal returns an Observable-session conflict/dependency report. Alt prepares a coordinated recommendation and requires its exact review; it does not execute the recommendation. Shift automatic mode additionally needs fully mediated participants, durable epochs, leases, pause/assignment/resume acknowledgements, and gates 4, 10, 15, 21, 24, and 27. It stays disabled until independent multi-agent acceptance evidence exists.

6.2 Workspace Command Bar

- **Claude, ChatGPT, VS Code, Terminal:** fixed focus/launch routes only. Missing approved applications are unavailable; no UI automation fallback may perform consequential work. Terminal selection preserves the exact workspace, focus-effect ownership is canonical, and repeated VS Code focus cycles only matching registered windows.
- **Commit:** local commit and PR upsert require reviewed exact scope. Merge and Direct Main are high-consequence and blocked by default.
- **GitHub:** navigation/status is read-only and target-bound. It never mutates a PR or workflow.
- **Compound:** Capture, Refresh, palette, and full loop have different mutation boundaries. Capture and Refresh use the inert manifest/review flow in section 5.6. The installed host-local executor applies only reviewed exact bytes, verifies before/after hashes and the accepted TargetSnapshot transition, and lets settled Refresh create only its exact private-index local commit. The provider supplies proposal material only and never performs the effect. Full `lfg` stays gate-disabled while its required gates are NOT ACCEPTED.
- **Run:** uses a host-attested recipe and exact managed process identity. Alt uses the split-binding flow in section 5.7 for an exact debug effect or durable managed Run child and never targets an arbitrary debugger, terminal, or PID.
- **Voice:** uses one explicit managed-session binding and one configured provider. Missing Claude support may not fall back to ChatGPT/Codex. Live microphone/privacy/routing acceptance is not claimed. All modifiers use the same durable effect ownership and settlement path, and targetful selection preserves the exact repository.
- **Pipeline:** status and traces are read-only; gates and reproduction use isolated recipes. Local and hosted observations preserve unavailable/stale distinctions, and Normal/Alt share one protected failure cursor. Repair Mission's typed deny-egress plan remains gate-disabled.
- **Shield:** results preserve passed/failed/unavailable/incomplete/not-run distinctions. Emergency Containment is bounded to brokered paths.
- **Sandbox:** `.venv` modes are not security isolation. Full Sandbox, rebuild, and Quarantine require the pinned Windows Sandbox or Linux Bubblewrap boundary and gates. `.venv` creation/rebuild transitions before the host action, which re-attests the exact interpreter identity before configuration and terminal creation.

7. High-Consequence Controls

Control	Chord/action	Default result	Why
Council	Shift / `council_automatic`	Blocked	Requires multi-agent mediation, epochs, leases, acknowledgements, and gates 4, 10, 15, 21, 24, 27.
Commit	Alt / `pr_merge`	Blocked	Requires exact head, current reviews/checks, final target guard, and gates 4, 6, 7, 15, 16, 17, 24.
Commit	Shift+Alt / `direct_main`	Blocked	Same merge gates plus repository allowlisting and final explicit authorization.
Compound	Shift+Alt / `lfg`	Blocked	May plan, edit, test, commit, push, open PR, and watch CI; requires gates 4, 5, 6, 15, 16, 19, 24, 27.
Pipeline	Ctrl+Shift+Alt / `repair_mission`	Blocked	Requires exact approved mutation scope, isolated reproduction, mediation, and gates 4, 5, 15, 16, 19, 24, 27.
Shield	Ctrl+Shift+Alt / `emergency_containment`	Blocked	Requires authenticated policy activation and gates 8 and 20. Release is never a chord.
Sandbox	Alt / `full_sandbox`	Unavailable or blocked	Requires the accepted OS boundary and gates 5, 12, 19, 27.
Sandbox	Shift+Alt / `rebuild_full_sandbox`	Unavailable or blocked	Also requires recovery/checkpoint gates 13 and 24.
Sandbox	Ctrl+Shift+Alt / `quarantine`	Unavailable or blocked	Requires accepted filesystem, credential, data, network, and resource enforcement.

Do not change `acceptance-gates.md`, a provider response, or a repository file to enable these controls. Only independently signed, current, target-scoped evidence plus reviewed host activation can satisfy the software policy, and the current register still marks every gate NOT ACCEPTED.

8. Update, Rollback, Uninstall, and Logs

8.1 Update

Use the new release's independently reviewed wrapper and exact trust values:

```
.\Update-B122.ps1 `
  -BundleRoot C:\reviewed\SXSY-B122-new-release `
  -ExpectedTrustDigest <reviewed-64-character-sha256> `
  -ExpectedSourceCommit <reviewed-40-character-commit>
```

Update imports a new immutable version, stops the candidate/prior process at defined boundaries, verifies health, and atomically changes the current pointer. On interruption, the next trusted lifecycle entry reconciles the journal before starting new work.

8.2 Rollback

Rollback selects only the authenticated `previous` version recorded by the installer. From the installed current bundle:

```
& "$env:LOCALAPPDATA\STRATHEX\switchyard\install\versions\<current-commit>\scripts\Rollback-B122.ps1"
```

If no exact previous version exists, rollback fails closed. Do not rename version directories or edit `install-state.json` by hand.

8.3 Uninstall product bytes while preserving state

Use the current installed uninstaller and allow its high-impact confirmation:

```
& "$env:LOCALAPPDATA\STRATHEX\switchyard\install\versions\<current-commit>\scripts\Uninstall-
B122.ps1"
```

The default removes only authenticated manifest-owned product bytes and the managed extension. Protected operational state is preserved. Use `-KeepVsCodeExtension` only when that is the reviewed intent.

8.4 Purge protected state

State purge is separate and destructive. It requires the exact canonical state root in `-ConfirmStateRoot`, the `-PurgeState` switch, and the exact `PURGE SXSY-B122 STATE <authenticated-state-root-id>` phrase in `-PurgeConfirmation`. Follow the uninstaller's displayed value; never guess or copy an identifier from another machine. Unknown files, reparse points, or changed identity preserve state and fail closed. The external tombstone must report completion before absence is claimed.

8.5 What to preserve for support

Before any manual recovery, preserve copies of these exact files if accessible:

- `install\install-state.json` and the authenticated lifecycle receipt;
- `logs\companion.stdout.log` and `logs\companion.stderr.log`;
- the bounded failure message and durable operation ID;
- the exact source commit and release bundle digest;
- the affected repository root and TargetSnapshot ID, without secret values;
- provider/route/helper identities and digests, not bearer secrets.

Never publish OS-protected secret envelopes, credential material, full voice transcripts, or unredacted failure output.

9. Firmware and Physical Device Boundary

The repository supplies signal-only firmware/protocol source, a 32-byte Raw-HID framing contract, manifest digest, host helper source, and a simulator. The host protocol covers a hello/ack handshake, connection epoch, counters, CRC, duplicate acknowledgement, and fail-closed status.

It does not establish:

- the actual keyboard VID/PID/report ID or installed firmware;
- collision-free physical key/chord emission;
- hot-plug, sleep/resume, multi-keyboard, Vial coexistence, or synthetic HID behavior;
- physical long-press timing, keycap readability, chord ergonomics, LEDs, or per-key lighting.

Do not flash firmware, remap the device, or manufacture keycaps from the documentation-order table. Physical position, legend artwork, fit, and device acceptance remain commissioning work.

10. Recovery Rules

10.1 Interrupted external work

If a command times out, crashes, or loses a provider response after dispatch:

1. Record the durable parent/child operation IDs.
2. Do not repeat the chord or restart the parent saga.
3. Use the operation status surface. Startup first asks the one authenticated durable controller installed for that provider to observe the exact child: the local gateway when installed for that provider, otherwise the configured out-of-process host.
4. If external state is ambiguous, leave the operation running/cancelling and non-retryable.
5. Retry only after a terminal result and explicit retry policy identify the failed predecessor.

The companion persists operation, action, session, provider, authority, TargetSnapshot, repository root, route, and external ID before dispatch. It does not fall back to raw provider network observation or redispatch merely because a response was lost. Native repository jobs are owned by the packaged Windows Job Object helper or exact Linux process-group/systemd boundary and settle only from protected terminal evidence. A poisoned runtime manifest may leave that one canonical repository scope ambiguous, but must not block status or recovery for healthy repository scopes.

10.2 Re-anchor versus rollback

Re-anchor asks an agent to stop safely, reread the task, inspect changes, and report. It never stashes, deletes, restores, or resumes automatically. A checkpoint record is a manual recovery guide with immutable references; it is not an automatic restore engine.

10.3 Containment incident

Containment activation persists a fence before best-effort revocation and cancellation hooks run. Treat unmediated shells, other apps, existing remote jobs, and credentials outside configured brokers as uncontrolled. Release requires the separate companion UI flow, reauthentication, exact target, reason, incident/rotation status, and current gate evidence. Never treat the Shield chord as a containment toggle.

11. Troubleshooting

11.1 Companion is unavailable

1. On Windows, run the authenticated installed launcher with `-WaitForHealth`. On Linux, run `systemctl --user status sxsy-b122.service` and inspect the bounded user journal for that unit.
2. Read the bounded companion stdout/stderr logs.
3. Verify the current version and payload were not modified. Startup rejects missing, added, or changed payload files.
4. Verify `%LOCALAPPDATA%\STRATHEX\switchyard\authenticated-pipe.json` on Windows or `${XDG_STATE_HOME:-$HOME/.local/state}/STRATHEX/switchyard/authenticated-pipe.json` on Linux exists, but do not edit or decrypt it manually.

5. If an install/update/rollback was interrupted, rerun the trusted installed launcher so lifecycle recovery runs.
6. If integrity or authentication fails, stop. Do not create an alternate pipe, token, TCP endpoint, or stdio fallback.

11.2 VS Code shows no target or an ambiguous target

1. Open the intended repository in its own VS Code window.
2. Run **Register/Refresh This VS Code Window**.
3. Run **Show Managed Sessions** and verify the exact canonical Git root.
4. Adopt a manually created agent terminal only through **Adopt Terminal Once**.
5. If several roots or sessions still match, use the picker. Cancel if the correct item cannot be proved.

11.3 A route is unavailable

This is often correct. Check the exact named prerequisite:

- fixed application path and executable digest for desktop focus;
- provider-host configuration, TLS pin, provider-module and host-effect-driver digests, companion-authenticated final-current callback, session handshake, and separately scoped host credential reference;
- repository recipe and executable digest for Run/Pipeline/Shield;
- exact HTTPS route, DNS pin, credential purpose, and containment state for network work;
- Windows feature plus pinned catalog/helper/lifecycle identities, or fixed `/usr/bin/bwrap` plus a typed pinned Linux recipe, for Sandbox;
- physical device installation and manifest match for Raw HID.

Never add an ambient `PATH`, proxy, credential helper, browser fallback, or free-form shell command to make the action run.

11.4 An operation will not cancel

Cancellation may be recorded while external observation is unavailable. Keep the exact operation visible. Verify the adapter supports cancel and observe for the same operation ID, external ID, action/route, session, authority, repository root, and TargetSnapshot. Native repository cancellation is addressed by exact durable operation or canonical repository scope and must use the managed Job Object path, never a reused PID. Provider observation must use the single configured authenticated durable controller; do not add a raw network fallback or submit a second provider request. An unmediated remote action may be genuinely uncontrolled.

11.5 A target changed during confirmation

The correct result is `incomplete` or `blocked`. Refresh repository state, review staged/unstaged/untracked/ignored/submodule/policy/tool inputs, and prepare a new exact review. Never reuse the old confirmation or receipt.

11.6 A high-consequence action is blocked

Check the named gates in the quick reference. The current project register says all 27 are NOT ACCEPTED. This is not a documentation error and must not be worked around. Collect independent acceptance evidence in a separate, authorized commissioning process.

11.7 Tests must not touch production data

Repository tests use isolated temporary roots, test databases, fixtures, or transactions that roll back. If a recipe cannot prove that production credentials, DSNs, hosts, databases, and data are absent, the recipe must remain unavailable. Do not run an ordinary terminal command as a substitute.

12. Deterministic Safe Demo



The repository-owned demo is a fixed, offline four-frame storyboard and HTML playback. It demonstrates preview, unavailable, blocked, and preserved outcomes without contacting hardware, providers, Git, credentials, scanners, or the network. It is explicitly simulated and supplies zero acceptance evidence.

Open `docs/demo/SXSY-B122-safe-demo.html` locally, or regenerate and verify all documentation artifacts:

```
python .\scripts\generate_b122_documentation.py
python .\scripts\generate_b122_documentation.py --check
python -m pytest .\docs\tests\test_b122_documentation.py -q
```

13. Evidence and Traceability

Claim area	Source contract	Local evidence boundary
22 controls / 65 assignments / 111 unassigned	`packages/protocol/generated/b122-control-map.json`	Documentation generator and protocol tests; no physical signal evidence.
Action, target, authority, confirmation, capability, gates	`packages/control-plane/src/command-catalog.ts`	Catalog/dispatch tests; no accepted external action.
Durable policy/receipts/recovery	`packages/control-plane/src/`	Isolated control-plane tests and protected temporary stores.
Windows/Linux companion and local IPC	`apps/windows-companion/src/` and `apps/linux-companion/src/`	Package/component tests and installed smoke; no same-user adversary acceptance.
VS Code operator surface	`apps/vscode-extension/src/`	VS Code-shaped tests and package build; no universal end-user/provider acceptance.
Provider host	`apps/provider-adapter-host/src/`	Deterministic fixtures and host contract tests; no bundled proprietary connector.
Firmware / Raw HID	`firmware/` and `packages/protocol/src/raw-hid.ts`	Compiler/protocol simulator checks; no device evidence.
Installation lifecycle	`deployment/windows/` and `deployment/linux/`	Isolated build/install/update/rollback/uninstall checks; no signed distribution claim.
Gate status	`docs/acceptance-gates.md`	All 27 NOT ACCEPTED.

The generated receipt at `docs/generated/SXSY-B122-documentation-manifest.json` records the source digests, exact 65 assignment/action/handler sets, output digests, and all gate statuses. The generator's `--check` mode fails when any generated output drifts.

14. Remaining External and Governance Work

14.1 Physical or target-host prerequisites

- Physical Raw-HID identity, firmware deployment, transport lifecycle, LEDs, legends, fit, chord ergonomics, and usability.
- Actual Windows Sandbox or Linux Bubblewrap availability and adversarial enforcement/cleanup on the target host.
- Real provider, GitHub, CI, scanner, Voice, credential, and Compound accounts, modules, routes, feeds, and current authorization.
- Signed distribution, independent OS/pipe/same-user review, and real install, update, rollback, uninstall, and recovery acceptance.
- Independent evidence for all 27 gates and deliberate activation of any high-consequence action.

14.2 Licensing and release authority

The repository currently declares `UNLICENSED`. File-level and project-level licensing, third-party redistribution permissions, trademark/keycap artwork review, and maintainer approval remain unresolved governance work. This manual does not choose a license or authorize distribution. The founding maintainer and appropriate legal reviewer must make that decision separately.

14.3 Historical records

The reconstructed interview notes are traceability material only. They must never be rewritten as a verbatim transcript. Earlier audit/roadmap documents that describe a foundation or candidate must remain clearly marked historical or superseded; current source and the present residual ledger control capability claims.

Preserved point-in-time records are `post-merge-adversarial-audit.md`, `software-completion-audit.md`, and `implementation-security-review.md`. They explain earlier findings and remediations; they are not the current capability, acceptance, provider, hardware, release, or licensing authority.

15. Acceptance Gate Register

Every gate is **NOT ACCEPTED**:

1. Hardware identity and chord uniqueness.
2. Multi-target integrity and picker-or-stop.
3. Honest unknown/stale/unavailable states.
4. Durable idempotency and crash reconciliation.
5. Test-data and production-credential isolation.
6. Scoped authority without bypass.
7. Immediate complete TargetSnapshot revalidation.
8. Containment honesty.
9. Claude/Codex provider parity.
10. Council failure cases.
11. Voice binding and redaction.
12. Quarantine isolation boundaries.
13. Checkpoint recoverability.
14. Physical usability and legends.
15. Mediated-only mutation enforcement.
16. Crash after every child side effect.
17. Snapshot invalidation inputs.
18. Prompt-injection resistance.
19. Credential isolation in high-risk paths.
20. Persistent containment and authenticated release.
21. Council epoch and lease safety.
22. Transport lifecycle and Raw HID coexistence.
23. Adapter degradation.
24. Partial-failure recovery.

25. Voice privacy and single-target routing.
26. Untrusted URL defense.
27. Budget, retry, concurrency, and cancellation.

No source change, test receipt, simulated demo, or keyboard event changes these statuses. See `docs/acceptance-gates.md` for the authoritative register.

Appendix A. SXSY-B122 Printable 22-Control / 65-Chord Reference

> *Generated from* `packages/protocol/generated/b122-control-map.json` *and* `packages/control-plane/src/command-catalog.ts`*. Do not edit this file by hand. Every acceptance gate is NOT ACCEPTED. A chord is intent, not authority.*

Print the final manual PDF on US Letter at 100% for a documentation reference. This table is not exact-scale keycap artwork or a physical placement guide.

Legend sheet

Bank	Key	Printed short legend	Settled chords
Left	1	Re-anchor	Normal
Left	2	Proof	Normal, Shift
Left	3	Option Provider	Normal, Shift
Left	4	Plan Expander	Normal
Left	5	Plan Approval	Normal
Left	6	Handoff	Normal, Shift, Alt, Shift+Alt
Left	7	Find Blocked Agent	Normal, Shift
Left	8	Find Finished Agent	Normal, Shift
Left	9	Authority Refresh	Normal, Shift
Left	10	Council	Normal, Shift, Alt
Upper	1	Claude	Normal, Shift, Alt
Upper	2	ChatGPT	Normal, Shift, Alt
Upper	3	VS Code	Normal, Shift
Upper	4	Terminal	Normal, Shift
Upper	5	Commit	Normal, Shift, Alt, Shift+Alt
Upper	6	GitHub	Normal, Shift, Alt, Shift+Alt
Upper	7	Compound	Normal, Shift, Alt, Shift+Alt
Upper	8	Run	Normal, Shift, Alt, Shift+Alt
Upper	9	Voice	Normal, Shift, Alt, Shift+Alt
Upper	10	Pipeline	Normal, Shift, Alt, Shift+Alt, Ctrl+Shift+Alt
Upper	11	Shield	Normal, Shift, Alt, Shift+Alt, Ctrl+Shift+Alt
Upper	12	Sandbox	Normal, Shift, Alt, Shift+Alt, Ctrl+Shift+Alt

Exact chord reference

The physical positions and production keycap artwork are not accepted. The numbering below is documentation order only.

#	Bank / key / chord	Action and settled behavior	Capability and safeguards
1	Agent Control Plane Re-anchor **Normal**	`reanchor` Safely re-anchor the active agent to scoped authority and evidence. Exact generated provider apply/observe routes preserve cooperative stop, discrepancy reporting, no resume, and no-replay recovery.	Mediation required Executor: `agent` capabilities registered+controllable; complete exact target; scoped authority; exact review/confirmation Handler: `b122.REANCHOR:normal`
2	Agent Control Plane Proof **Normal**	`proof` Request evidence-based self-verification. Durable typed result; missing mediation or configuration is unavailable/blocked.	Mediation required Executor: `agent` capabilities registered+observable; complete exact target; scoped authority Handler: `b122.PROOF:normal`
3	Agent Control Plane Proof **Shift**	`proof_adversarial` Request a cross-backend adversarial review. Durable typed result; missing mediation or configuration is unavailable/blocked.	Mediation required Executor: `agent` capabilities registered+observable; complete exact target; scoped authority; exact review/confirmation Handler: `b122.PROOF:shift`
4	Agent Control Plane Option Provider **Normal**	`option_provider` Request five options and one recommendation. Durable typed result; missing mediation or configuration is unavailable/blocked.	Mediation required Executor: `agent` capabilities registered+controllable; complete exact target; scoped authority Handler: `b122.OPTION_PROVIDER:normal`
5	Agent Control Plane Option Provider **Shift**	`option_packet` Request one numbered packet for pending decisions. Durable typed result; missing mediation or configuration is unavailable/blocked.	Mediation required Executor: `agent` capabilities registered+controllable; complete exact target; scoped authority Handler: `b122.OPTION_PROVIDER:shift`
6	Agent Control Plane Plan Expander **Normal**	`plan_expander` Request fifteen ranked creative alternatives. Durable typed result; missing mediation or configuration is unavailable/blocked.	Mediation required Executor: `agent` capabilities registered+controllable; complete exact target; scoped authority Handler: `b122.PLAN_EXPANDER:normal`
7	Agent Control Plane Plan Approval **Normal**	`plan_approval` Route an already-approved plan to the selected session. The reviewed expected TargetSnapshot transition and durable provider child effects are observed; contradiction or target churn reconciles.	Mediation required Executor: `agent` capabilities mediated; complete exact target; scoped authority; exact review/confirmation Handler: `b122.PLAN_APPROVAL:normal`
8	Agent Control Plane Handoff **Normal**	`handoff` Prepare a planner-to-coding-agent context transfer. Durable typed result; missing mediation or configuration is unavailable/blocked.	Mediation required Executor: `agent` capabilities registered; complete exact target; scoped authority Handler: `b122.HANDOFF:normal`
9	Agent Control Plane Handoff **Shift**	`handoff_queue` Queue a blocker or decision packet. Durable typed result; missing mediation or configuration is unavailable/blocked.	Mediation required Executor: `agent` capabilities registered; complete exact target; scoped authority Handler: `b122.HANDOFF:shift`
10	Agent Control Plane Handoff **Alt**	`handoff_immediate` Focus the planner with the packet. Provider workflow: deliver/focus the exact reviewed packet.	Route-only Executor: `agent` capabilities registered; complete exact target; scoped authority Handler: `b122.HANDOFF:alt`
11	Agent Control Plane Handoff **Shift+Alt**	`handoff_routing_default` Toggle queued versus immediate-focus handoff. Reviewed durable routing-default change.	Route-only Executor: `adapter` complete exact target; exact review/confirmation Handler: `b122.HANDOFF:shift_alt`

#	Bank / key / chord	Action and settled behavior	Capability and safeguards
12	Agent Control Plane Find Blocked Agent **Normal**	`focus_blocked_agent` Focus the next verified blocked managed agent. Durable typed result; missing mediation or configuration is unavailable/blocked.	Mediation required Executor: `agent` capabilities registered+observable; complete exact target; scoped authority Handler: `b122.FIND_BLOCKED_AGENT:normal`
13	Agent Control Plane Find Blocked Agent **Shift**	`blocker_queue` Open the blocker queue. The picker exposes only verified-blocked sessions for the exact authorized repository; no exact match stops.	Route-only Executor: `adapter` complete exact target Handler: `b122.FIND_BLOCKED_AGENT:shift`
14	Agent Control Plane Find Finished Agent **Normal**	`focus_finished_agent` Focus the next verified finished managed agent. Durable typed result; missing mediation or configuration is unavailable/blocked.	Mediation required Executor: `agent` capabilities registered+observable; complete exact target; scoped authority Handler: `b122.FIND_FINISHED_AGENT:normal`
15	Agent Control Plane Find Finished Agent **Shift**	`done_queue` Open the done queue. The picker exposes only verified-finished sessions for the exact authorized repository; no exact match stops.	Route-only Executor: `adapter` complete exact target Handler: `b122.FIND_FINISHED_AGENT:shift`
16	Agent Control Plane Authority Refresh **Normal**	`authority_refresh` Replay existing scoped authority to the active agent. Durable typed result; missing mediation or configuration is unavailable/blocked.	Mediation required Executor: `agent` capabilities registered+controllable; complete exact target; scoped authority Handler: `b122.AUTHORITY_REFRESH:normal`
17	Agent Control Plane Authority Refresh **Shift**	`authority_broadcast` Prepare a reviewed individualized broadcast. Two-phase exact-recipient broadcast; cancellation, refusal, churn, and host failure settle once through the canonical receipt path.	Mediation required Executor: `agent` capabilities registered+controllable; complete exact target; scoped authority; exact review/confirmation Handler: `b122.AUTHORITY_REFRESH:shift`
18	Agent Control Plane Council **Normal**	`council_report` Report verified cross-agent conflicts and dependencies. Durable typed result; missing mediation or configuration is unavailable/blocked.	Mediation required Executor: `council` capabilities registered+observable; complete exact target Handler: `b122.COUNCIL:normal`
19	Agent Control Plane Council **Shift**	`council_automatic` Automatic partitioning and isolation (Council automatic mode). Blocked unless independently accepted gates and all exact prerequisites are current.	Disabled pending gates Executor: `council` capabilities registered+observable+controllable+mediated; complete exact target; scoped authority; exact review/confirmation; gates 4, 10, 15, 21, 24, 27 Handler: `b122.COUNCIL:shift`
20	Agent Control Plane Council **Alt**	`council_recommend` Propose one coordinated plan for explicit approval. Durable typed result; missing mediation or configuration is unavailable/blocked.	Mediation required Executor: `council` capabilities registered+observable; complete exact target; exact review/confirmation Handler: `b122.COUNCIL:alt`
21	Workspace Command Bar Claude **Normal**	`open_claude` Open or focus Claude Desktop. Fixed local route; a missing approved surface is unavailable.	Route-only Executor: `adapter` installed fixed host route; no repository target Handler: `b122.CLAUDE:normal`
22	Workspace Command Bar Claude **Shift**	`fresh_claude` Open a fresh Claude surface. Fixed local route; a missing approved surface is unavailable.	Route-only Executor: `adapter` exact review/confirmation Handler: `b122.CLAUDE:shift`
23	Workspace Command Bar Claude **Alt**	`open_managed_claude_terminal` Open a managed Claude Code terminal in VS Code. Durable typed result; missing mediation or configuration is unavailable/blocked.	Mediation required Executor: `adapter` capabilities registered; complete exact target; scoped authority Handler: `b122.CLAUDE:alt`
24	Workspace Command Bar ChatGPT **Normal**	`open_chatgpt` Open or focus ChatGPT Desktop. Fixed local route; a missing approved surface is unavailable.	Route-only Executor: `adapter` installed fixed host route; no repository target Handler: `b122.CHATGPT:normal`

#	Bank / key / chord	Action and settled behavior	Capability and safeguards
25	Workspace Command Bar ChatGPT **Shift**	`fresh_chatgpt` Open a fresh ChatGPT surface. Fixed local route; a missing approved surface is unavailable.	Route-only Executor: `adapter` exact review/confirmation Handler: `b122.CHATGPT:shift`
26	Workspace Command Bar ChatGPT **Alt**	`open_managed_codex_terminal` Open a managed Codex terminal in VS Code. Durable typed result; missing mediation or configuration is unavailable/blocked.	Mediation required Executor: `adapter` capabilities registered; complete exact target; scoped authority Handler: `b122.CHATGPT:alt`
27	Workspace Command Bar VS Code **Normal**	`focus_vscode` Focus the last-used VS Code window. Focus or cycle only registered VS Code windows matching the exact repository; ambiguity never chooses another workspace.	Route-only Executor: `adapter` installed fixed host route; no repository target Handler: `b122.VSCODE:normal`
28	Workspace Command Bar VS Code **Shift**	`new_vscode` Open a new VS Code window. Fixed local route; a missing approved surface is unavailable.	Route-only Executor: `adapter` exact review/confirmation Handler: `b122.VSCODE:shift`
29	Workspace Command Bar Terminal **Normal**	`focus_terminal` Open or focus an external terminal. One companion-owned fixed terminal focus route settles through the canonical durable host-action path.	Route-only Executor: `adapter` installed fixed host route; no repository target Handler: `b122.TERMINAL:normal`
30	Workspace Command Bar Terminal **Shift**	`focus_vscode_terminal` Open or focus the VS Code integrated terminal. Create or reuse only a terminal rooted at the exact authorized repository; never select an arbitrary active terminal.	Route-only Executor: `adapter` complete exact target Handler: `b122.TERMINAL:shift`
31	Workspace Command Bar Commit **Normal**	`local_commit` Create a scoped local commit after exact review. Durable typed result; missing mediation or configuration is unavailable/blocked.	Mediation required Executor: `git` capabilities mediated; complete exact target; scoped authority; exact review/confirmation Handler: `b122.COMMIT:normal`
32	Workspace Command Bar Commit **Shift**	`pr_upsert` Push and create or update the exact pull request after exact review. Durable typed result; missing mediation or configuration is unavailable/blocked.	Mediation required Executor: `git` capabilities mediated; complete exact target; scoped authority; exact review/confirmation; outbound containment fence Handler: `b122.COMMIT:shift`
33	Workspace Command Bar Commit **Alt**	`pr_merge` Broker-enforced exact-head PR merge. Blocked unless independently accepted gates and all exact prerequisites are current.	Disabled pending gates Executor: `git` capabilities mediated; complete exact target; scoped authority; exact review/confirmation; gates 4, 6, 7, 15, 16, 17, 24; outbound containment fence Handler: `b122.COMMIT:alt`
34	Workspace Command Bar Commit **Shift+Alt**	`direct_main` Broker-enforced guarded Direct Main. Blocked unless independently accepted gates and all exact prerequisites are current.	Disabled pending gates Executor: `git` capabilities mediated; complete exact target; scoped authority; exact review/confirmation; gates 4, 6, 7, 15, 16, 17, 24; outbound containment fence Handler: `b122.COMMIT:shift_alt`
35	Workspace Command Bar GitHub **Normal**	`github_home` Open the repository home. Read-only route for one exact target; ambiguity stops or opens a picker.	Route-only Executor: `adapter` complete exact target Handler: `b122.GITHUB:normal`
36	Workspace Command Bar GitHub **Shift**	`github_pr` Open the current-branch PR or PR list. Read-only route for one exact target; ambiguity stops or opens a picker.	Route-only Executor: `adapter` complete exact target Handler: `b122.GITHUB:shift`
37	Workspace Command Bar GitHub **Alt**	`github_issues` Open repository issues. Read-only route for one exact target; ambiguity stops or opens a picker.	Route-only Executor: `adapter` complete exact target Handler: `b122.GITHUB:alt`

#	Bank / key / chord	Action and settled behavior	Capability and safeguards
38	Workspace Command Bar GitHub **Shift+Alt**	`github_actions` Open the current-branch Actions run or list. Read-only route for one exact target; ambiguity stops or opens a picker.	Route-only Executor: `adapter` complete exact target Handler: `b122.GITHUB:shift_alt`
39	Workspace Command Bar Compound **Normal**	`compound_capture` Capture one durable learning. Review one proposal-derived inert manifest, then apply only its exact host-local bytes and verify the host-observed target transition.	Mediation required Executor: `automation` capabilities mediated; complete exact target; scoped authority; exact review/confirmation Handler: `b122.COMPOUND:normal`
40	Workspace Command Bar Compound **Shift**	`compound_refresh` Preview learning-store refresh scope. Apply reviewed host-local file effects, then create the exact private-index local commit and verify the resulting transition.	Mediation required Executor: `automation` capabilities mediated; complete exact target; scoped authority; exact review/confirmation Handler: `b122.COMPOUND:shift`
41	Workspace Command Bar Compound **Alt**	`compound_palette` Open the Compound workflow palette. Read-only route for one exact target; ambiguity stops or opens a picker.	Route-only Executor: `adapter` complete exact target Handler: `b122.COMPOUND:alt`
42	Workspace Command Bar Compound **Shift+Alt**	`lfg` Guarded full lfg loop. Exact typed child planning is implemented, but dispatch remains blocked without current activation, gate evidence, and configured artifacts.	Disabled pending gates Executor: `automation` capabilities mediated; complete exact target; scoped authority; exact review/confirmation; gates 4, 5, 6, 15, 16, 19, 24, 27; outbound containment fence Handler: `b122.COMPOUND:shift_alt`
43	Workspace Command Bar Run **Normal**	`run_target` Run the selected configured target. Durable typed result; missing mediation or configuration is unavailable/blocked.	Mediation required Executor: `execution` capabilities mediated; complete exact target; scoped authority Handler: `b122.RUN:normal`
44	Workspace Command Bar Run **Shift**	`debug_target` Debug the selected configured target. Durable typed result; missing mediation or configuration is unavailable/blocked.	Mediation required Executor: `execution` capabilities mediated; complete exact target; scoped authority Handler: `b122.RUN:shift`
45	Workspace Command Bar Run **Alt**	`stop_managed_run` Stop a managed run process. Stop only the selected exact durable debug or managed Run effect; an ambiguous result remains reconciliation-visible instead of replaying.	Mediation required Executor: `execution` capabilities mediated; complete exact target; scoped authority; exact review/confirmation Handler: `b122.RUN:alt`
46	Workspace Command Bar Run **Shift+Alt**	`launch_web_app` Start a managed web app and offer its verified local URL. Durable typed result; missing mediation or configuration is unavailable/blocked.	Mediation required Executor: `execution` capabilities mediated; complete exact target; scoped authority; exact review/confirmation Handler: `b122.RUN:shift_alt`
47	Workspace Command Bar Voice **Normal**	`open_voice` Open the two-way voice center. One canonical durable host-action lifecycle owns and settles the Voice Center effect.	Route-only Executor: `adapter` installed fixed host route; no repository target Handler: `b122.VOICE:normal`
48	Workspace Command Bar Voice **Shift**	`switch_voice_provider` Choose the Claude or ChatGPT provider. Fixed local route; a missing approved surface is unavailable.	Route-only Executor: `adapter` exact review/confirmation Handler: `b122.VOICE:shift`
49	Workspace Command Bar Voice **Alt**	`bind_voice_session` Bind voice only to a selected managed VS Code agent. Bind through the canonical durable authority, confirmation, exact managed-session, effect, and receipt lifecycle.	Mediation required Executor: `agent` capabilities registered; complete exact target; scoped authority; exact review/confirmation Handler: `b122.VOICE:alt`
50	Workspace Command Bar Voice **Shift+Alt**	`voice_session_picker` Open the voice session picker. Open the exact-repository picker; selection performs the reviewed durable single-session binding.	Route-only Executor: `adapter` complete exact target Handler: `b122.VOICE:shift_alt`

#	Bank / key / chord	Action and settled behavior	Capability and safeguards
51	Workspace Command Bar Pipeline **Normal**	`pipeline_status` Show non-mutating pipeline status. Read-only route for one exact target; ambiguity stops or opens a picker.	Route-only Executor: `adapter` complete exact target Handler: `b122.PIPELINE:normal`
52	Workspace Command Bar Pipeline **Shift**	`pipeline_local_gate` Run a local gate through the test-environment broker. Durable typed result; missing mediation or configuration is unavailable/blocked.	Mediation required Executor: `test_environment` capabilities mediated; complete exact target; scoped authority Handler: `b122.PIPELINE:shift`
53	Workspace Command Bar Pipeline **Alt**	`pipeline_failure_trace` Diagnose the selected failure. Normal and Alt share one protected failure cursor over bounded local and hosted evidence; absent routes remain unavailable.	Mediation required Executor: `adapter` complete exact target Handler: `b122.PIPELINE:alt`
54	Workspace Command Bar Pipeline **Shift+Alt**	`pipeline_clean_reproduction` Reproduce cleanly through the test-environment broker. Durable typed result; missing mediation or configuration is unavailable/blocked.	Mediation required Executor: `test_environment` capabilities mediated; complete exact target; scoped authority Handler: `b122.PIPELINE:shift_alt`
55	Workspace Command Bar Pipeline **Ctrl+Shift+Alt**	`repair_mission` Isolated Repair Mission. Exact deny-egress sandbox and typed child planning are implemented, but dispatch remains blocked without current activation and gate evidence.	Disabled pending gates Executor: `automation` capabilities mediated; complete exact target; scoped authority; exact review/confirmation; gates 4, 5, 15, 16, 19, 24, 27 Handler: `b122.PIPELINE:ctrl_shift_alt`
56	Workspace Command Bar Shield **Normal**	`shield_change_scan` Scan current changes using configured coverage. Durable typed result; missing mediation or configuration is unavailable/blocked.	Mediation required Executor: `security` capabilities mediated; complete exact target; scoped authority Handler: `b122.SHIELD:normal`
57	Workspace Command Bar Shield **Shift**	`shield_codebase_audit` Run a deeper codebase and history audit. Durable typed result; missing mediation or configuration is unavailable/blocked.	Mediation required Executor: `security` capabilities mediated; complete exact target; scoped authority Handler: `b122.SHIELD:shift`
58	Workspace Command Bar Shield **Alt**	`shield_supply_chain` Run configured supply-chain audit coverage. Durable typed result; missing mediation or configuration is unavailable/blocked.	Mediation required Executor: `security` capabilities mediated; complete exact target; scoped authority Handler: `b122.SHIELD:alt`

#	Bank / key / chord	Action and settled behavior	Capability and safeguards
59	Workspace Command Bar Shield **Shift+Alt**	`release_shield` Prepare an exact-revision Release Shield receipt. Durable typed result; missing mediation or configuration is unavailable/blocked.	Mediation required Executor: `security` capabilities mediated; complete exact target; scoped authority Handler: `b122.SHIELD:shift_alt`
60	Workspace Command Bar Shield **Ctrl+Shift+Alt**	`emergency_containment` Containment release or emergency containment workflow. Blocked unless independently accepted gates and all exact prerequisites are current.	Disabled pending gates Executor: `containment` capabilities mediated; complete exact target; scoped authority; exact review/confirmation; gates 8, 20 Handler: `b122.SHIELD:ctrl_shift_alt`
61	Workspace Command Bar Sandbox **Normal**	`venv` Create or enter a repository-local virtual environment. Complete the repo-local environment transition, then re-attest exact interpreter path, digest, and file identity before host selection.	Mediation required Executor: `execution` capabilities mediated; complete exact target; scoped authority Handler: `b122.SANDBOX:normal`
62	Workspace Command Bar Sandbox **Shift**	`rebuild_venv` Atomically rebuild and swap the repository virtual environment after exact review; preserve recovery state on failure. Atomically rebuild the repo-local environment, then re-attest exact interpreter path, digest, and file identity before host selection.	Mediation required Executor: `execution` capabilities mediated; complete exact target; scoped authority; exact review/confirmation Handler: `b122.SANDBOX:shift`

#	Bank / key / chord	Action and settled behavior	Capability and safeguards
63	Workspace Command Bar Sandbox **Alt**	`full_sandbox` Create or enter a Full Sandbox. Blocked unless independently accepted gates and all exact prerequisites are current.	Disabled pending gates Executor: `isolation` capabilities mediated; complete exact target; scoped authority; exact review/confirmation; gates 5, 12, 19, 27 Handler: `b122.SANDBOX:alt`
64	Workspace Command Bar Sandbox **Shift+Alt**	`rebuild_full_sandbox` Rebuild a Full Sandbox. Blocked unless independently accepted gates and all exact prerequisites are current.	Disabled pending gates Executor: `isolation` capabilities mediated; complete exact target; scoped authority; exact review/confirmation; gates 5, 12, 13, 19, 24, 27 Handler: `b122.SANDBOX:shift_alt`
65	Workspace Command Bar Sandbox **Ctrl+Shift+Alt**	`quarantine` Quarantine. Blocked unless independently accepted gates and all exact prerequisites are current.	Disabled pending gates Executor: `isolation` capabilities mediated; complete exact target; scoped authority; exact review/confirmation; gates 5, 12, 19, 27 Handler: `b122.SANDBOX:ctrl_shift_alt`

Intentionally unassigned surface

The eight documented modifier classes produce 176 possible slots. Exactly 65 are settled above; the remaining 111 are intentionally unassigned. In particular, Ctrl, Ctrl+Shift, and Ctrl+Alt have no settled behaviors. The dispatcher must reject those combinations rather than inventing a fallback.

Appendix B. Source Index

- Accepted specification and adversarial review: `docs/SXSY-B122-complete-key-specification-and-audit.md`
- Reconstructed interview record: `docs/historical-interview-notes-reconstructed.md`
- Implementation-oriented adversarial index: `docs/adversarial-review.md`
- Gate register: `docs/acceptance-gates.md`
- Software mechanism/evidence audit: `docs/software-only-completion-audit.md`
- Residual ledger: `docs/software-residual-ledger.md`
- Historical post-merge adversarial audit: `docs/post-merge-adversarial-audit.md`
- Historical software completion audit: `docs/software-completion-audit.md`
- Historical implementation security review: `docs/implementation-security-review.md`
- Operational safety model: `docs/operational-safety-model.md`
- Provider adapter contract: `docs/provider-adapter-protocol.md`
- Windows lifecycle guide: `docs/windows-release-and-installation.md`
- Linux lifecycle guide: `docs/linux-release-and-installation.md`
- Canonical protocol source: `packages/protocol/src/b122.ts`
- Generated control map: `packages/protocol/generated/b122-control-map.json`
- Command catalog: `packages/control-plane/src/command-catalog.ts`
- Documentation generator: `scripts/generate_b122_documentation.py`